# Gli operatori logici: AND, OR, NOT

Lezione 9 — 1IOT, SST-ICT-01 — Operatori logici, tabelle di verità, condizioni composte nell'if, prime esercitazioni

05/10/2026

## 1 Da dove ripartiamo

**Nota**

Parlando delle **condizioni** da mettere dentro l'`if`, il passo successivo è capire **quali tipologie di condizioni** possiamo costruire. Finora abbiamo visto tre categorie di operatori:

- **operatori matematici**: `+`, `-`, `*`, `/` e `%` (modulo);
- **operatori di confronto**: maggiore `>`, minore `<`, maggiore uguale `>=`, minore uguale `<=`, uguaglianza `==` e disuguaglianza `!=`;
- **operatore di assegnazione** `=`, che copia (modifica) un numero dentro una variabile.

Oggi aggiungiamo una quarta categoria, che state un po' vedendo anche in matematica, ma qui la guardiamo a livello pratico, per quello che ci serve: gli **operatori logici**.

**Definizione**

Gli **operatori logici** sono operatori che danno come risposta sempre e solo **vero o falso**. Anche i loro ingressi sono di tipo **vero** o falso. Sono usati in tantissime situazioni: in questo corso li useremo sia nella parte di programmazione (dentro le **condizioni** dell'`if`) sia nella parte di logica ed elettronica (le **porte logiche**, che ritroverete con **Arduino**).

## 2 Vero e falso come 1 e 0

**Definizione**

Nelle tabelle di questa lezione al posto di **F** (**falso**) si scrive **0** e al posto di **V** (**vero**) si scrive **1**. È solo una questione di simbologia usata dagli informatici, coerente con i numeri binari già visti. D'ora in poi useremo quasi esclusivamente **0 e 1**. È "la prima e unica volta" in cui la tabella viene scritta con F e V: alla lavagna la tabella della OR è stata scritta prima così, e accanto con 0 e 1.

| a | b | Q |
|---|---|---|
| F | F | F |
| F | V | V |
| V | F | V |
| V | V | V |

| a | b | Q |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

Intestazione scritta alla lavagna: "OPERATORI LOGICI (VERO o FALSO)" — AND e OR hanno 2 **input** (A, B) e 1 **output** (Q), il NOT 1 **input** (A) e 1 **uscita** (Q).

In C vale una regola importantissima: lo **zero** è **falso**, qualunque altro numero è **vero**. Non solo l'1: anche 7, 21, 3, e anche i numeri negativi come $-3$ sono veri, perché sono diversi da **zero**.

## 3 L'operatore AND

**Definizione**

L'operatore **AND** richiede (almeno) due ingressi, che chiamiamo `a` e `b`, e fornisce un'uscita, che chiamiamo `Q`. In C si scrive con la **doppia e commerciale**:

```
Q = a && b;
```

**Definizione**: la AND dà come uscita **vero solo se entrambi** gli ingressi sono veri.

| a | b | Q = a && b |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Nota**

Il simbolo **&** (e commerciale) si trova sulla **tastiera** italiana sopra il **6** (`Shift + 6`). È un simbolo usato parecchio dai programmatori. La tabella qui sopra va copiata sul quaderno: la useremo spesso.

**Esempio**

**La sicurezza delle presse (esempio storico raccontato in aula).** Tanti anni fa capitava che gli operai che lavoravano alle presse, per distrazione, perdessero le mani: appoggiavano la mano sulla pressa, la avviavano, e la pressa gliela schiacciava. Come si è risolto il problema? **Con l'operatore AND**: si obbliga l'operatore a premere contemporaneamente **due pulsanti** distanti tra loro, in modo che non si possa fare con una mano sola. Se tutte e due le mani sono occupate sui pulsanti, nessuna mano può trovarsi sotto la pressa. La macchina ha il consenso di funzionare solo se **entrambi i pulsanti** sono premuti: è esattamente una **porta AND**. Un problema di sicurezza risolto alla radice spendendo quasi niente: un pulsante la pressa lo aveva già, ne basta un secondo. (Questa soluzione esiste davvero e si chiama **comando a due mani**: è regolata da una norma tecnica europea, la EN ISO 13851.)

**Esempio**

**Un altro esempio di AND**: vengo promosso sicuramente se ho **tutti** i voti positivi, in **tutte** le materie. Basta una sola materia negativa perché la condizione "tutti positivi" diventi **falsa**.

**Definizione**

La **porta AND** ha anche un **simbolo grafico**, usato negli schemi elettrici: è fatto come una specie di **mezzo cerchio**, con due ingressi da un lato e l'uscita dall'altro. In un circuito reale agli ingressi vengono applicate davvero delle tensioni (e quindi delle correnti) che rappresentano 0 o 1, e la porta risponde in uscita con 0 o 1 seguendo la tabella di verità.

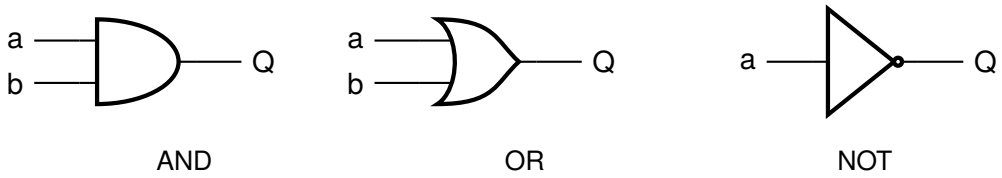



Figura 1: I simboli delle tre porte logiche viste in aula: AND (mezzo cerchio), OR, NOT (triangolo con pallino).

## 4 L'operatore OR

**Definizione**

Anche l'operatore **OR** ha due ingressi e un'uscita. In C si scrive con la **doppia barra verticale**:

```
Q = a || b;
```

**Definizione**: la OR dà come uscita **falso solo se entrambi** gli ingressi sono falsi. In tutti gli altri casi dà **vero**.

| a | b | Q = a \|\| b |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Nota**

Il **maggiore** > sta sul tasto in basso a sinistra, accanto alla Z, insieme al minore **<**: si ottiene con `Shift`. **Maggiore o uguale** si scrive con due caratteri, prima il maggiore e poi l'uguale: **>=** (e allo stesso modo <=). La barra verticale `|` sulla **tastiera** italiana si trova **in alto a sinistra**, sullo stesso tasto del **backslash** `\` (a sinistra dell'1): si ottiene con `Shift + \`.

**Definizione**

La **porta AND** e la porta OR si dicono **complementari**: se guardate le due definizioni, la frase cambia solo per due parole.

- la AND dà l'uscita **vera** solo se entrambi gli ingressi sono **veri**;
- la OR dà l'uscita **falsa** solo se entrambi gli ingressi sono **falsi**.

Una lavora in maniera esattamente opposta all'altra. Queste sono le due definizioni preferite

dal docente.

**Attenzione**

La OR ha anche una **seconda definizione**, che dice esattamente la stessa cosa e che potreste trovare su ChatGPT, su YouTube o nei video in giro: la OR dà vero se **almeno un ingresso** è vero. "Almeno uno" vuol dire che vanno bene anche **tutti e due**: con a = 1 e b = 1 la OR dà 1. Esiste una porta diversa, che dà **vero** solo quando gli ingressi sono **diversi** tra loro (e 0 quando sono **entrambi** a 1): è un'altra porta logica, la **XOR**, che vedremo più avanti. Non va confusa con la OR.

**Nota**

**AND come prodotto, OR come somma.** La seconda definizione serve a far vedere un'altra cosa. Grosso modo la tabella della AND è equivalente alla **moltiplicazione** tra i due ingressi: $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, e solo $1 \cdot 1 = 1$. La OR invece è quasi come la **somma**: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$; e $1 + 1$ in logica fa ancora 1 (non può esistere un "2": il risultato è solo **vero** o falso).

## 5 L'operatore NOT

**Definizione**

L'ultimo **operatore logico** che ci serve si chiama **NOT**. A differenza degli altri due ha **un solo ingresso** (`a`) e una sola uscita (`Q`). In C si scrive con il **punto esclamativo** davanti alla variabile:

```
Q = !a;
```

`!a` si legge "NOT a": prendi lo stato di `a` (**vero** o falso) e su `Q` metti **esattamente il contrario**. Se `a` era **vera**, `Q` è **falsa**; se `a` era **falsa**, `Q` è vera. Il NOT **inverte lo stato dell'ingresso**. La sua tabella di verità è molto più piccola:

| a | Q = !a |
|---|---|
| 0 | 1 |
| 1 | 0 |

**Esempio**

**L'ombrello, in versione logica.** "Se devo uscire **e non** piove, allora esco senza ombrello; altrimenti porto l'ombrello." Si può dire anche: "se devo uscire e **non è vero** che non piove", cioè se piove, "allora esco con l'ombrello". Due NOT uno dentro l'altro si annullano.

```
if (devoUscire && !piove)
{
    printf("Esco senza ombrello");
}
else
{
```

```
    printf("Porto l'ombrello");
}
```

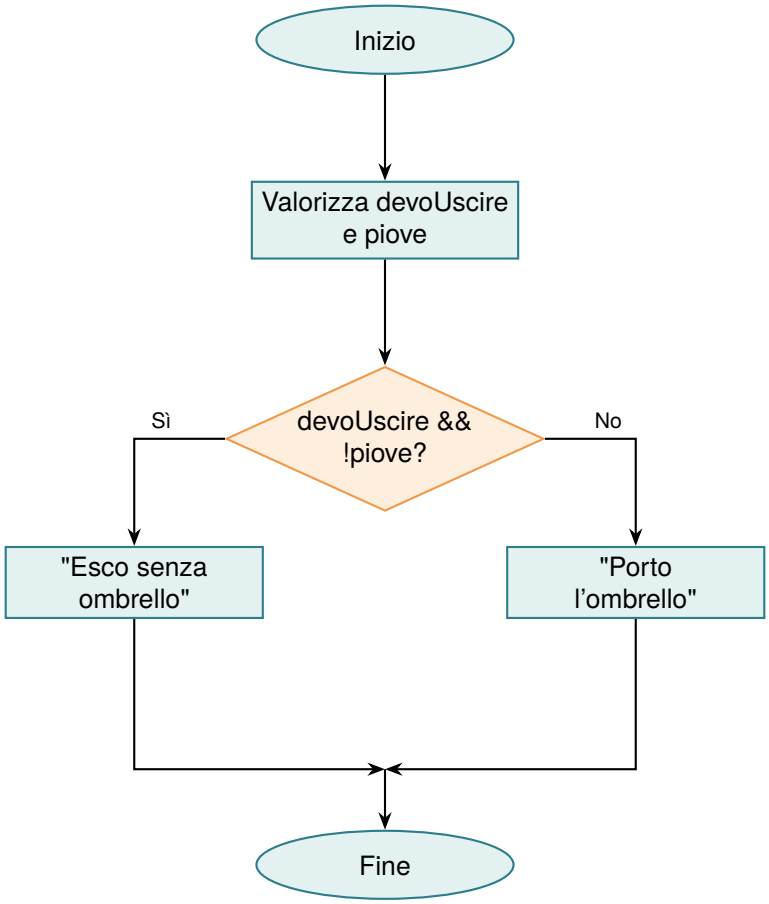



Figura 2: Diagramma di flusso dell'esempio dell'ombrello: una sola decisione, ma con una condizione composta da AND e NOT.

**Nota**

**Riassunto da tenere a fianco**: AND (&&) vero **solo se** tutti e due veri; OR (||) falso **solo se** tutti e due falsi; NOT (!) **inverte**. Sono i tre **operatori logici** che dobbiamo proprio imparare, perché il programmatore vive di **AND**, **OR** e **NOT**: combinandoli possiamo far fare alla macchina delle cose solo quando succede qualcosa di particolare.

## 6 Operatori logici dentro l'if: leggere il codice

**Esempio**

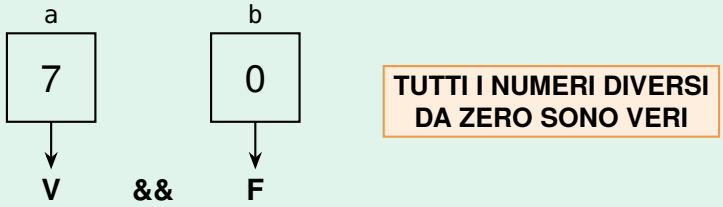
**Primo esempio alla lavagna.** Cosa stampa questo codice?

```
char a;
char b;
a = 7;
b = 0;
```

```c
if (a && b)
{
    printf("Buongiorno");
}
else
{
    printf("Buonasera");
}
```

Le risposte date in aula sono state le più varie: "se sono le 7 scrivi buongiorno" (ma in quale punto il codice fa una valutazione sull'ora? da nessuna parte), "la **variabile** `a` è buongiorno e la `b` buonasera", "stampa la **variabile** a", "stampa la prima perché viene prima", "non stampa niente perché non ci sono dati". **La macchina non inventa niente**: bisogna leggere il codice riga per riga.

1. `char a;` crea una **variabile**; `char b;` ne crea un'altra.
2. `a = 7;` l'operatore di assegnazione copia il numero 7 dentro `a`: ogni volta che chiedo `a`, la macchina mi risponde 7.
3. `b = 0;` dentro `b` c'è il numero 0. Attenzione: 0 non è "niente", è un numero come tutti gli altri.
4. `if (a && b)`: la macchina fa la AND tra `a` e `b`. Qualunque numero tranne 0 è **vero**, quindi `a` (7) è **vero**; `b` (0) è **falso**, perché lo 0 è l'unico numero che la macchina interpreta come **falso**.
5. Vero AND **falso**, cioè $1 \;\&\&\; 0$: nella tabella della AND dà **0, falso**.
6. Se la condizione è **falsa** l'`if-else` esegue il blocco `else`.




Lo schema disegnato alla lavagna: `a = 7` è VERO, `b = 0` è FALSO; sopra la **condizione** il docente ha scritto $1 \;\&\&\; 0$, e due frecce dalla **condizione** ai due **`printf`**.
Il codice, scritto così, stampa sempre "Buonasera". Un `if-else` esegue sempre uno solo dei due blocchi, mai entrambi.

### Esempio

**Come faccio a fargli stampare "Buongiorno"?** La risposta giusta non è "assegnare un numero anche a `b`" (un numero, lo 0, ce l'ha già): bisogna assegnare a `b` un **numero diverso da 0**. Va bene anche $-3$? **Sì**: non abbiamo **mai** detto che solo i positivi sono veri. Tutti i numeri diversi da **zero** sono veri, $-3$ compreso. Con `b = -3;` (variante scritta alla lavagna) la condizione diventa **vero** AND vero e il programma stampa "Buongiorno".

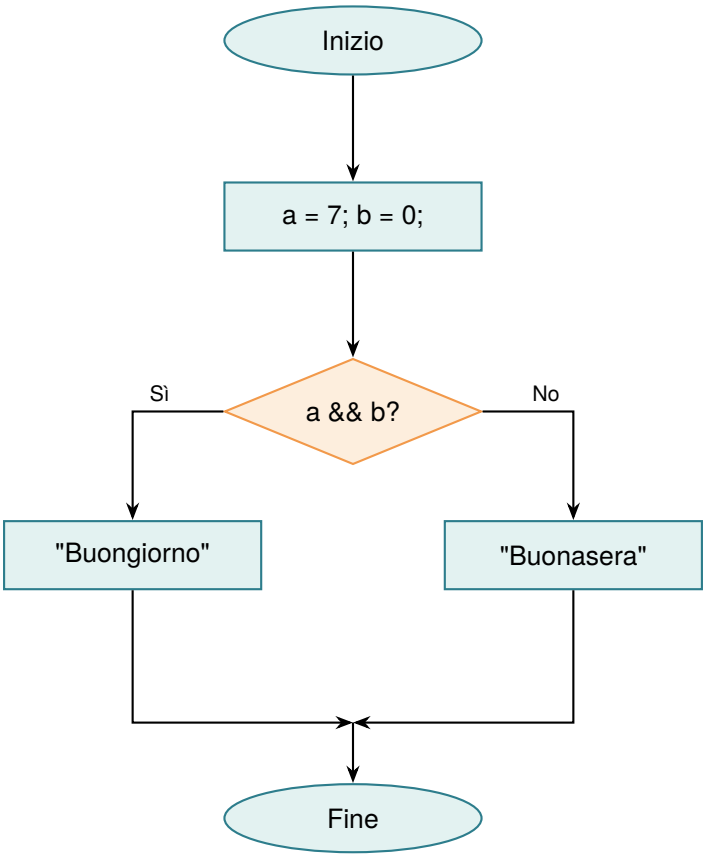



Figura 3: Lo schema di tutti gli esempi "Buongiorno/Buonasera" di questa sezione: cambia solo la condizione nel rombo.

**Esempio**

**Stesso codice, con la OR**: `if (a || b)` con `a = 7` e `b = 0`. Vero OR falso dà **vero**: stampa "Buongiorno". Un allievo ha risposto "perché c'è prima il 7": non è una **questione di ordine**. Se scambio `a` e `b` il risultato non cambia; per una OR basta che **almeno uno** sia **vero**.
Quando stampa "Buonasera"? Solo se **tutti e due** sono falsi, cioè **entrambi** valgono 0: falso OR falso dà falso.
Altro caso: `a = 7` e `b = 3`. Bisogna fare subito la conversione: 7 è **vero**, 3 è vero. Nella tabella della OR, $1 \,||\, 1$ dà **vero**, quindi esegue il primo **blocco**. Se metto `b = 0`, **vero** OR falso fa ancora vero. L'unico modo per fargli stampare "Buonasera" è che anche `a` valga 0.

**Esempio**

**Tre variabili.** Con `if (a && b && c)`, se `a` vale 0 la **condizione** è falsa **qualunque cosa** valgano b e c: lo 0 dentro una AND "annichilisce tutto".
Caso più difficile, discusso a lungo in aula, scritto alla lavagna così:

```
char a;
char b;
char c;
a = 7;
b = 2;
c = 5;
```

```c
if (a && (b && !c))
{
    printf("Buongiorno");
}
else
{
    printf("Buonasera");
}
```

a, b e c valgono tutte un numero diverso da **zero** (sono tutte vere). La c col **punto esclamativo** davanti viene negata e diventa **falsa**. Risposta sentita in aula: "due su tre sono vere, quindi è vero". **È sbagliato**: non si ragiona "a maggioranza". Bisogna guardare come si combinano a livello logico: dentro una AND basta un solo **falso** perché tutto diventi **falso**. La c negata rende **falsa** la parentesi `(b && !c)`, e questo **falso** manda a zero anche la AND esterna con a. Quindi la condizione è **falsa** e il programma stampa "Buonasera".
Poco prima alla lavagna c'era la variante con la OR dentro la parentesi, `if (a && (b || !c))`: qui `(b || !c)` è **vero** perché basta b **vero**, quindi tutta la condizione è **vera** e stampa "Buongiorno". Stessi valori, operatore diverso nella parentesi, risultato opposto.

**Esempio**

**Il NOT su una sola variabile.** Con `a = 7` e la **condizione** `if (!a)`: a vale 7, tutti i numeri diversi da **zero** sono veri, quindi a è **vero**. Mettendoci davanti il **punto esclamativo** lo stato si scambia e diventa **falso**: il programma stampa "Buonasera". Se togliessi il `!`, a sarebbe **vero** e stamperebbe "Buongiorno".

**Definizione**

**Cosa può stare tra le parentesi dell'if.** Dentro le parentesi c'è qualcosa che vale **vero** o falso: se è **vero** si esegue il primo blocco (tra le prime **graffe**), se è falso il blocco `else`. Come realizzi quel **vero** o falso sta a te. Per esempio potrei scrivere `b = 7 * 3;`: dentro b va 21, che è **vero** perché diverso da zero.
Attenzione ai termini: si dice che la condizione è **vera** o **falsa**, non "positiva" o "negativa". Non confondiamo i termini.

# 7 Esercitazione al computer: onlinegdb

**Nota**

I programmi si scrivono sul sito **onlinegdb.com**, lo stesso usato per i primi programmi. Consiglio del docente: crearsi un **account** su onlinegdb e salvarci tutti i codici. Per ogni nuovo esercizio fate un **progetto nuovo** ("New"), scrivetelo da capo, e salvatelo: non modificate il vecchio codice "rattoppando" pezzi, altrimenti perdete l'esercizio precedente e fate confusione.

**Attenzione**

Chi è stato assente o non ha seguito le lezioni precedenti prima di scrivere codice deve **leg-**

**gersi le dispense** (in particolare A07 e A08 sotto "Lavori del corso" su **Classroom**): "sennò è come mettersi su una macchina senza aver fatto la patente". Dove qualcosa non torna, si scrive una mail al docente e lo si riprende insieme passo passo.

## 7.1 Il compito V02 su Classroom

**Nota**

Il compito si chiama "**V02 – Esercizi con Operatori Logici**" (100 punti); il documento allegato è "V02 – Esercizi Operatori Logici". Le indicazioni passo passo sono scritte solo nel primo esercizio: gli altri sono simili e vanno adattati. I testi visibili a **schermo** durante la lezione (gli esercizi dal 5 al 10 non sono stati mostrati):

1. **Esame e Patente**. Scrivere un programma in C che richieda all'utente di inserire l'età e la presenza della patente (1 per sì, 0 per no). Solo se l'età è **maggiore o uguale** a 18 **E** la patente è uguale a 1, il programma stamperà "Puoi guidare", altrimenti stamperà "Non puoi guidare".
2. **Sconto Cinema**. Scrivere un programma in C che richieda all'utente di inserire la propria età. Se l'età è minore di 12 **OPPURE** maggiore o uguale a 65, il programma stamperà "Sconto attivo", altrimenti stamperà "Tariffa intera".
3. **Allarme Disattivato**. Scrivere un programma in C che richieda all'utente lo stato dell'allarme (1 per attivo, 0 per disattivato). Usando l'operatore NOT (`!`), se l'allarme **NON** è attivo il programma stamperà "Casa sicura", altrimenti stamperà "Allarme in corso".
4. **Voto Valido**. Scrivere un programma in C che richieda all'utente di inserire un voto da 1 a 10. Se il voto è **maggiore o uguale** a 1 **E** minore o uguale a 10, il programma stamperà "Voto valido", altrimenti stamperà "Voto non valido".

**Nota**

Nei listati di questa dispensa è riportato solo il contenuto del programma. A **schermo** il codice completo sta dentro `int main() { ...}`, con `#include <stdio.h>` in cima e `return 0;` come ultima istruzione prima della graffa di chiusura.

## 7.2 Esercizio guidato: promosso o bocciato con due voti

**Esempio**

**Testo**: scrivere un programma in C che chieda all'utente di inserire **due voti in centesimi**. Solo se **entrambi** i voti sono maggiori o uguali a 60 viene stampata la scritta "promosso", altrimenti "bocciato".
Svolgimento fatto insieme alla lavagna:

- i voti vanno da 0 a 100, quindi andrebbe bene anche un `char`; usiamo `int` per chiarezza (chi ha usato `char` va bene lo stesso, ma vedi l'avviso più sotto);
- per ogni voto: un `printf` che fa la domanda e uno `scanf` che acquisisce il dato;

- lasciate uno **spazio** alla fine del messaggio (`"Inserisci il primo voto: "`): senza, il numero che l'utente scrive compare attaccato al testo. Non è un **errore**, è solo brutto da vedere;
- i due punti alla fine del messaggio sono pura estetica: tutto quello che sta tra le virgolette viene stampato così com'è, alla macchina non importa cosa c'è scritto.

```c
int voto1;
int voto2;

printf("Inserisci il primo voto: ");
scanf("%d", &voto1);
printf("Inserisci il secondo voto: ");
scanf("%d", &voto2);

if ((voto1 >= 60) && (voto2 >= 60))
{
    printf("Promosso");
}
else
{
    printf("Bocciato");
}
```

Prove fatte in aula: primo voto 65, secondo voto 80 → "Promosso"; 55 e 60 → "Bocciato".

**Attenzione**

**Errore proposto in aula**: scrivere `if (voto1 && voto2)`. Non funziona: 70 è **vero** perché diverso da 0, ma anche 30 è **vero** perché diverso da 0, quindi la condizione è **quasi sempre vera** e stamperebbe "Promosso" anche con 30. La **condizione** da mettere è che i voti siano **maggiori o uguali a 60**: a sinistra della AND la prima **condizione** (`voto1 >= 60`), a destra la seconda (`voto2 >= 60`). Se la prima è **vera**, è il primo "**vero**" della AND; poi si controlla il secondo voto.

**Attenzione**

**Errori visti girando tra i banchi**:

- **punto e virgola dopo l'if**: `if (...);` è sbagliato. L'`if` **non vuole** il **punto e virgola**, ma richiede le **graffe**; il compilatore segnala ad esempio che si aspettava una parentesi tonda prima della graffa;
- **graffe messe dove capita**: la struttura dell'**`if`** è una sola, `if (condizione) { ...} else { ...}`. Non si può aprire la graffa prima dell'**`if`**, né chiudere quella del **`main`** al posto di quella dell'**`if`**. "Bisogna essere precisissimi su queste cose: non possiamo fare quello che vogliamo";
- tra le graffe ci va **tutto** quello che si deve fare se la condizione è **vera**: stampare, comunicare, elaborare;
- un allievo aveva dichiarato i voti come **`char`** e il programma non funzionava: con `scanf("%d", ...)` la **variabile** deve essere un **`int`** (`%d` si aspetta un **intero**, non un `char`). Cambiando in `int` è andato.

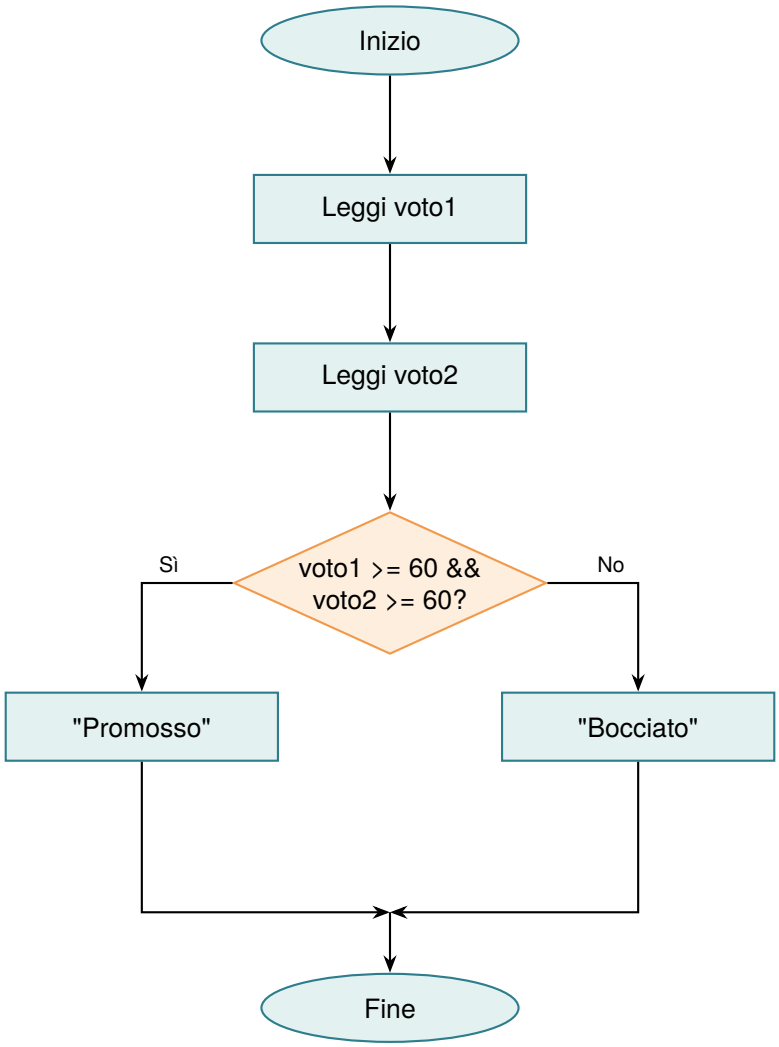



Figura 4: Diagramma di flusso dell'esercizio 1.

## 7.3 Ripasso al volo: stampare il contenuto di una variabile

**Definizione**

Tutto quello che sta tra i due apici di **`printf`** viene stampato a monitor senza curarsi di cosa c'è scritto: posso scrivere anche che 3 + 1 fa 7 e me lo stampa lo stesso. È una **stringa**, non ha nessun valore di calcolo. **`printf`** però può stampare anche il **contenuto di una variabile**: si aggiunge la **variabile** dopo il testo, separata da una virgola, e nel testo si mette **`%d`** nel punto in cui deve comparire il numero.

```
int anni;
anni = 42;
printf("Bortolani ha %d anni", anni);
```

All'esecuzione stampa `Bortolani ha 42 anni`: al posto di `%d` viene messo il numero contenuto nella variabile.

## 7.4 Esercizio 1 del compito: Esame e Patente

**Esempio**

**Testo**: scrivere un programma in C che chieda all'utente di inserire l'**età** e la **presenza della patente** (1 per sì, 0 per no). Solo se l'età è **maggiore o uguale** a 18 **e** la patente è uguale a 1 stampa "Puoi guidare", altrimenti "Non puoi guidare". Svolto insieme alla lavagna:

1. creare due **variabili** `int`: una per l'età e una per l'informazione se ha o meno la patente;
2. chiedere l'età e acquisirla; poi chiedere la patente e acquisirla, specificando all'utente cosa scrivere ("1 = sì, 0 = no");
3. decidere con un **`if-else`** e una AND.

Le indicazioni scritte nel commento in cima al file: "crea due **variabili** di tipo int, una la chiami eta e l'altra patente. Usi printf e scanf per acquisire e memorizzare l'età della persona e specifica che alla richiesta *hai la patente?* la risposta deve essere 0 per dire no, e 1 per dire sì. A questo punto puoi fare l'if inserendo le due **condizioni** e usare E (and) per verificare che siano vere entrambe. Se la condizione è **vera** usi printf per stampare *Puoi guidare*, mentre nell'else usi un printf per stampare *Non puoi guidare*". Un allievo ha proposto di chiedere "sei maggiorenne o minorenne?": è una domanda diversa, la **consegna** chiede di inserire l'età.

```c
int eta;
int patente;

printf("Inserisci la tua eta': ");
scanf("%d", &eta);
printf("Hai la patente? (1=si, 0=no) ");
scanf("%d", &patente);

if ((eta >= 18) && (patente == 1))
{
    printf("Puoi guidare");
}
else
{
    printf("Non puoi guidare");
}
```

**Attenzione**

**Errori visti nelle consegne dell'esercizio 2**:

- **lettere accentate** nel nome della variabile (`età`): non si possono usare caratteri speciali nei nomi delle **variabili**, creano problemi su certi linguaggi. Si scrive `eta`. Dentro le virgolette del **`printf`** invece l'accento si può mettere, perché è solo testo (anche se in console le lettere accentate a volte si stampano male: per questo nel codice sopra si usa l'apostrofo);
- **due domande contemporaneamente**: non si chiede età e patente nello stesso **`printf`**. La struttura è sempre **`printf`** e `scanf`, poi di nuovo **`printf`** e `scanf`: domandi l'età e acquisisci l'età, domandi la patente e acquisisci la patente. Ogni dato va nella **sua variabile**;

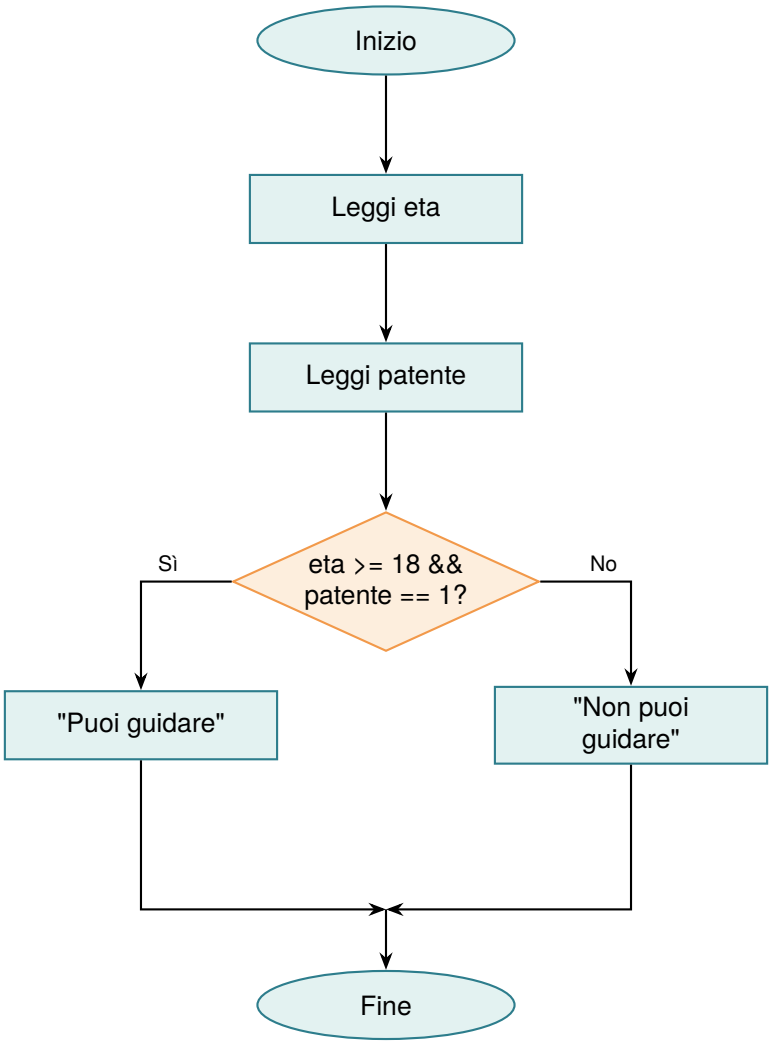



Figura 5: Diagramma di flusso dell'esercizio 2.

- **dimenticare lo scanf**: senza `scanf("%d", &patente);` la variabile non viene **mai** valorizzata;
- **= al posto di ==**: `patente = 1` mette l'1 dentro `patente`, non chiede se vale 1. Per chiederlo servono **due uguali**;
- **graffe**: tutto il codice deve stare dentro il contenitore delle graffe del `main`; la graffa dell'**`if`** va chiusa prima dell'`else`, non dopo;
- un **punto e virgola** mancante (nel caso visto, alla riga 15);
- due **variabili** per la stessa età (`eta1`, `eta2`): l'età è un dato solo, serve una **variabile** sola.

## 7.5 Esercizio 2 del compito: Sconto Cinema

**Esempio**

L'esercizio chiede di applicare lo sconto a chi ha **meno di 12 anni** oppure **almeno 65** (**maggiore o uguale** a 65; a voce, girando tra i banchi, è stato detto "maggiore di 65", ma fa fede il testo). Domanda del docente a chi aveva scritto una AND: "come può essere che tu abbia meno di 12 anni **e anche** più di 65?" Nessuno può soddisfare entrambe le **condizioni** insieme, quindi con la AND lo sconto non verrebbe **mai** applicato. Qui serve la **OR**:

```c
if ((eta < 12) || (eta >= 65))
{
    printf("Sconto attivo");
}
else
{
    printf("Tariffa intera");
}
```

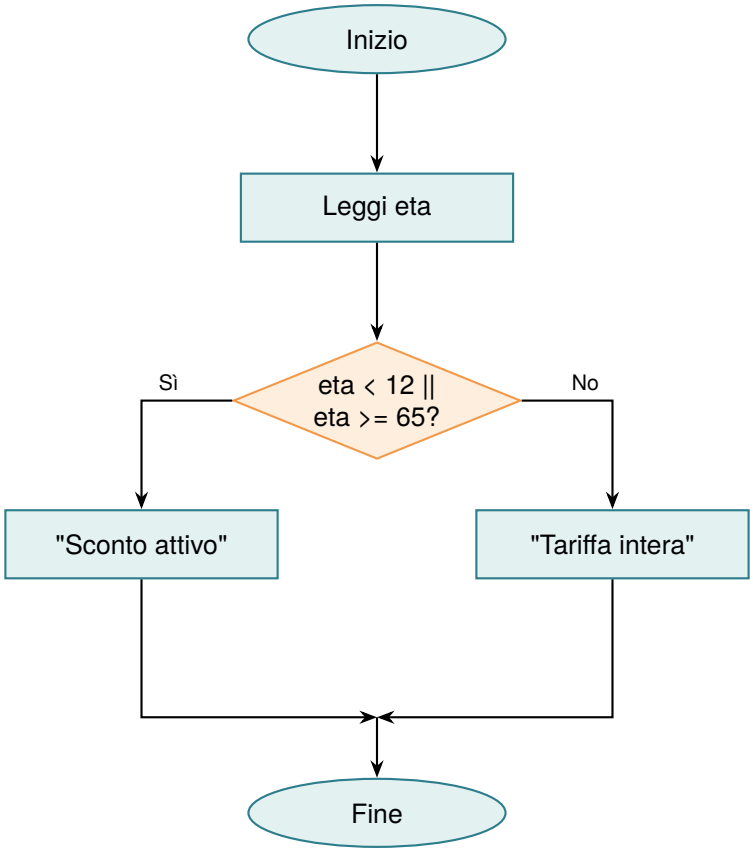



Figura 6: Diagramma di flusso dell'esercizio dello sconto: qui serve la OR, non la AND.

## 8 Consegne su Classroom e regole per i compiti

**Nota**

Su **Classroom** è stato assegnato un compito con **dieci esercizi** ("V02 – Esercizi Operatori Logici"): ogni esercizio ha il testo scritto come commento in cima al file, e si svolge su **onlinegdb**. Man mano che finite un esercizio lo **consegnate**: il docente li controlla e vede dove state sbagliando. Come consegnare uno screenshot del codice: si fa la cattura dello schermo, si apre **Paint**, si incolla con `Ctrl+V`, si salva il file (la parte nera non serve, si può ritagliare), poi su **Classroom** "Aggiungi" o "Crea" → "File" → "Sfoglia" e si allega. Lo stream di **Classroom** non è il posto dove esercitarsi: i materiali sono in "Lavori del corso".

**Attenzione**

Gli esercizi non finiti in aula vanno **finiti a casa**: c'è una settimana di tempo, fino a **lunedì prossimo**, in concomitanza con la **verifica**. Il docente ha spiegato perché vuole le **consegne**: vuole vedere se siete capaci di programmare.

- Chi non fa i **compiti** a casa e non consegna: durante la **verifica** non viene aiutato.
- Chi **consegna** esercizi in parte giusti e in parte sbagliati ha lavorato e ci ha provato: durante la **verifica** può anche essere aiutato.
- Chi **consegna** esercizi tutti perfetti e poi alla **verifica** va malissimo, probabilmente se li è fatti fare da qualcun altro: si vede subito. L'importante è essere pronti per la **verifica**.

Per dubbi si può scrivere alla mail del docente.

**Nota**

**Verifica**: proposta per **lunedì 12 ottobre**, circa un'ora. Comprende **tutti gli argomenti** fatti finora: "fra un anno e mezzo dovete comunque sapere queste cose: non è che finito un argomento lo si può dimenticare, è una costruzione continua". Il giorno dopo questa lezione è previsto un ripasso generale.

## 9 Cosa ricordare di questa lezione

- Gli **operatori logici** danno sempre e solo **vero o falso**; in tabella si usano 0 (falso) e 1 (vero).
- In C **0 è falso**, ogni altro numero è **vero**, anche i negativi.
- **AND** `&&`: vero solo se **entrambi** veri (come la moltiplicazione). Esempio: comando a due mani delle presse.
- **OR** `||`: falso solo se **entrambi** falsi; equivalente: vero se almeno uno è vero (come la somma, con $1 + 1 = 1$).
- AND e OR sono **complementari**.
- **NOT** `!`: un solo ingresso, **inverte** lo stato.

- Nelle **condizioni composte** non si ragiona “a maggioranza” (“due su tre”): si applicano le tabelle.
- Una **condizione** su un intervallo di valori (“**entrambi** i voti almeno 60”) si scrive con due confronti uniti da **&&**, non con `voto1 && voto2`.
- = assegna, == confronta. L’`if` non vuole il **punto e virgola** e ha una sola struttura di **graffe** possibile.